

数据结构

基本概念

线性表

栈、队列和数组

考试大纲

一、基本概念

(一) 数据结构的基本概念 (去年新增)

(二) 算法的基本概念 (去年新增)

二、线性表

(一) 线性表的基本概念

(二) 线性表的实现

1. 顺序存储

2. 链式存储

(三) 线性表的应用

三、栈、队列和数组

(一) 栈和队列的基本概念

(二) 栈和队列的顺序存储结构

(三) 栈和队列的链式存储结构

(四) 多维数组的存储

(五) 特殊矩阵的压缩存储

(六) 栈、队列和数组的应用

| 1.1 408考纲要求综述

绪论部分的核心考点

- 掌握数据结构的基本概念（数据元素、逻辑结构、物理结构）。
- 理解算法的基本特征。
- **高频考点：**熟练掌握算法时间复杂度和空间复杂度的分析与计算。

提示：绪论通常以单项选择题形式出现，分值约2-4分，但它是解答后续 408 综合应用题（算法设计题）复杂度分析的基础。

| 1.2 数据结构的基本概念

- **数据 (Data)**: 客观事物的符号表示, 是所有能输入到计算机中并被程序处理的符号的总称。
- **数据元素 (Data Element)**: 数据的基本单位, 通常作为一个整体进行考虑和处理。
- **数据项 (Data Item)**: 构成数据元素的不可分割的最小单位。
- **数据对象 (Data Object)**: 性质相同的数据元素的集合, 是数据的一个子集。

包含关系: 数据 > 数据对象 > 数据元素 > 数据项

1.3 逻辑结构的三要素

逻辑结构是指数据元素之间的逻辑关系，与计算机存储无关。

- **集合结构**：元素同属一个集合，无其他关系。
- **线性结构**：一对一关系（除首尾元素外）。
- **树形结构**：一对多关系。
- **图状/网状结构**：多对多关系。

408选择题常考区分逻辑结构与物理结构的名词。

例如：“栈”、“队列”是逻辑结构；而“顺序栈”、“链表”涉及物理实现。

| 1.4 物理结构（存储结构）

数据结构在计算机中的表示称为物理结构或存储结构。

- **顺序存储**：逻辑上相邻的元素物理上也相邻（依赖数组实现）。优势是随机访问。
- **链式存储**：逻辑上相邻的元素物理上不一定相邻，通过指针建立联系。避免空间碎片。
- **索引存储**：建立附加的索引表来标识节点的地址。
- **散列存储 (Hash)**：根据元素的关键字直接计算出该元素的存储地址。

| 1.5 算法的基本特征

算法是对特定问题求解步骤的一种描述。

1. **有穷性**：有限步骤后必须结束。
2. **确定性**：每一步骤必须有确切的定义，无二义性。
3. **可行性**：算法中执行的任何计算步都是可以被分解为基本的可执行的操作步。
4. **输入**：有零个或多个输入。
5. **输出**：有一个或多个输出。

好算法的目标

正确性、可读性、健壮性、高效率与低存储量需求。

| 1.6 渐进时间复杂度分析

分析算法执行时间随问题规模 n 增长的趋势: $T(n) = O(f(n))$

- 加法规则: $T(n) = T_1(n) + T_2(n) = O(\max(f(n), g(n)))$
- 乘法规则: $T(n) = T_1(n) \times T_2(n) = O(f(n) \times g(n))$

常见时间复杂度按数量级递增排序 (必背):

$$O(1) < O(\log_2 n) < O(n) < O(n \log_2 n) < O(n^2) < O(n^3) < O(2^n) < O(n!) < O(n^n)$$

1.7 代码演示：时间复杂度计算

示例 1：线性对数阶

```
int i = 1, count = 0;
while (i <= n) {
    i = i * 2;
    count++;
}
```

分析：每次循环 i 翻倍，设执行 k 次，则 $2^k \leq n$ ，即 $k \leq \log_2 n$ 。复杂度： $O(\log n)$

示例 2：嵌套循环

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        x++;
    }
}
```

分析：内层执行次数为 $0 + 1 + 2 + \dots + (n - 1) = \frac{n(n-1)}{2}$ 。
复杂度： $O(n^2)$

| 1.8 空间复杂度分析

空间复杂度 $S(n) = O(g(n))$ ，指算法运行过程中所使用的辅助空间的大小。

易错点剖析

- **原地工作 (In-place):** 算法所需的辅助空间为常量，即 $S(n) = O(1)$ 。408 算法题常要求原地操作。
- **递归程序的空间复杂度:** 等于递归调用的最大深度。例如普通的递归求解阶乘空间复杂度为 $O(n)$ 。

1.9 递归时间复杂度分析



$$T(n) = 2T(n/2) + cn$$

16. 【2022 统考真题】下列程序段的时间复杂度是 ()。

```
int sum=0;
for(int i=1;i<n;i*=2)
    for(int j=0;j<i;j++)
        sum++;
```

A. $O(\log n)$

B. $O(n)$

C. $O(n \log n)$

D. $O(n^2)$

16. B

对于单层循环如 `for(j=0;j<=i;j++) sum++;`，可以直接数出执行次数为 i ，因此可将多层循环转换成多个并列的单层循环，且列出每个单层循环如下（假设 t 为循环变量的幂次）：

循环变量 i	单层循环语句	单层循环执行次数
1	<code>for(j=0;j<1;j++)</code>	1
2^1	<code>for(j=0;j<2;j++)</code>	2
2^2	<code>for(j=0;j<2^2;j++)</code>	4
...	...	...
2^t	<code>for(j=0;j<2^t;j++)</code>	2^t

进入外层循环的条件是 $i < n$ ，当循环结束时，循环变量的幂次 t 满足 $2^t < n \leq 2^{t+1}$ 。总执行次数 $T = 1 + 2^1 + 2^2 + \dots + 2^t = 2^{t+1} - 1$ ，即 $n - 1 \leq T$ 且 $T < 2n - 1$ ，所以时间复杂度为 $O(n)$ 。

已知某递归算法的运行时间满足以下递推关系：

$$T(n) = 2T(n - 1) + 1, \quad T(0) = 1$$

请问该算法的时间复杂度为：

- A. $O(n)$
- B. $O(n \log n)$
- C. $O(2^n)$
- D. $O(n^2)$

C

$$T(n) = 2T(n - 1) + 1$$

$$T(n) = 2[2T(n - 2) + 1] + 1 = 2^2T(n - 2) + 2 + 1$$

$$T(n) = 2^2[2T(n - 3) + 1] + 2 + 1 = 2^3T(n - 3) + 2^2 + 2 + 1$$

$$T(n) = 2^kT(n - k) + (2^{k-1} + 2^{k-2} + \dots + 2^1 + 1)$$

终止条件：

当 $n - k = 0$ 时，即 $k = n$ ，递归停止。此时 $T(0) = 1$ ：

$$T(n) = 2^nT(0) + (2^{n-1} + 2^{n-2} + \dots + 1)$$

$$T(n) = 2^n \cdot 1 + (2^n - 1)$$

$$T(n) = 2 \cdot 2^n - 1 = 2^{n+1} - 1$$

2.1 线性表的定义与ADT

线性表是具有**相同数据类型**的 n ($n \geq 0$) 个数据元素的**有限序列**。

- 除第一个元素外，每个元素有且仅有一个直接前驱。
- 除最后一个元素外，每个元素有且仅有一个直接后继。

```
// 线性表抽象数据类型基本操作概览  
InitList(&L); // 初始化表  
Length(L); // 求表长  
LocateElem(L, e); // 按值查找  
ListInsert(&L, i, e); // 插入操作  
ListDelete(&L, i, &e); // 删除操作
```

2.2 顺序表 (SqList) 结构体定义

顺序表在内存中分配一块连续的空间。408 中通常有两种定义方式：静态分配与动态分配。

静态分配 (数组固定)

```
#define MaxSize 50
typedef struct {
    ElemType data[MaxSize];
    int length;
} SqList;
```

动态分配 (指针灵活)

```
#define InitSize 100
typedef struct {
    ElemType *data;
    int MaxSize, length;
} SeqList;
```

2.3 顺序表的初始化 (动态分配)

动态分配依赖于 C 语言的 malloc 函数（位于 stdlib.h）。

```
void InitList(SeqList &L) {  
    // 申请一块连续的存储空间  
    L.data = (ElemType *)malloc(InitSize * sizeof(ElemType));  
    if (L.data == NULL) {  
        // 内存分配失败处理...  
    }  
    L.length = 0;  
    L.MaxSize = InitSize;  
}
```

注：C++ 中可直接使用 new 和 delete，408 考试阅卷时均可接受。

2.4 顺序表的插入操作代码

在顺序表 L 的第 i 个位置 ($1 \leq i \leq L.length + 1$) 插入新元素 e 。

```
bool ListInsert(SqList &L, int i, ElemType e) {  
    if (i < 1 || i > L.length + 1) return false; // 边界检查  
    if (L.length >= MaxSize) return false; // 空间已满  
  
    for (int j = L.length; j >= i; j--) {  
        L.data[j] = L.data[j-1]; // 元素后移, 腾出位置 (注意下标)  
    }  
    L.data[i-1] = e; // 插入元素  
    L.length++;  
    return true;  
}
```

时间复杂度：平均需要移动 $\frac{n}{2}$ 个元素，为 $O(n)$ 。

2.5 顺序表的删除操作代码

删除顺序表 L 中第 i 个位置的元素，并用引用变量 e 返回。

```
bool ListDelete(SqList &L, int i, ElemType &e) {  
    if (i < 1 || i > L.length) return false; // 边界检查  
  
    e = L.data[i-1]; // 保存被删元素  
    for (int j = i; j < L.length; j++) {  
        L.data[j-1] = L.data[j]; // 元素前移填补空缺  
    }  
    L.length--;  
    return true;  
}
```

时间复杂度：平均需要移动 $\frac{n-1}{2}$ 个元素，为 $O(n)$ 。

| 2.6 单链表 (Singly Linked List) 结构定义

单链表通过指针实现，不需要连续内存，但失去了随机访问特性。

```
typedef struct LNode {  
    ElemType data; // 数据域  
    struct LNode *next; // 指针域  
} LNode, *LinkList;
```

理解别名： LNode 强调这是一个节点，LinkList 强调这是一个链表的头指针。两者在底层是等价的，但在语义上有所区分，这是 408 标准答案的代码风格。

| 2.7 头结点 vs 头指针

头指针 (Head Pointer)

始终指向链表的第一个节点。若有头结点，则指向头结点；若无，则指向第一个存有数据的节点。无论链表是否为空，头指针都不为空（除了无头结点的空链表）。

带头结点的链表

在首元节点之前附加的一个节点，其 next 指向真实的第一个元素。

优点：使首元节点的插入/删除操作与其他节点代码一致，无需判断表是否为空来特殊处理。

| 2.8 单链表的初始化代码 (带头结点)

为单链表分配头结点，并将 next 置空。

```
bool InitList(LinkList &L) {  
    L = (LNode *)malloc(sizeof(LNode)); // 分配头结点  
    if (L == NULL) return false; // 内存不足  
    L->next = NULL; // 头结点指针域置空  
    return true;  
}
```

判断空表的条件：L->next == NULL。

2.9 建立单链表：头插法代码

头插法将新节点始终插入到头结点之后，因此最终链表的顺序与输入顺序**相反**。

```
LinkedList List_HeadInsert(LinkedList &L) {  
    LNode *s;  
    int x;  
    L = (LinkedList)malloc(sizeof(LNode));  
    L->next = NULL; // 必须先置空  
    scanf("%d", &x);  
    while (x != -1) { // 输入 -1 表示结束  
        s = (LNode *)malloc(sizeof(LNode));  
        s->data = x;  
        s->next = L->next; // 第一步：新节点指向原来的首元节点  
        L->next = s; // 第二步：头结点指向新节点  
        scanf("%d", &x);  
    }  
    return L;  
}
```

2.10 建立单链表：尾插法代码

增加一个尾指针 r，始终指向链表的最后一个节点。输入与生成顺序相同。

```
LinkList List_TailInsert(LinkList &L) {  
    int x;  
    L = (LinkList)malloc(sizeof(LNode));  
    LNode *s, *r = L; // r 为表尾指针  
    scanf("%d", &x);  
    while (x != -1) {  
        s = (LNode *)malloc(sizeof(LNode));  
        s->data = x;  
        r->next = s; // 尾部插入新节点  
        r = s; // 尾指针后移  
        scanf("%d", &x);  
    }  
    r->next = NULL; // 最后必须置空  
    return L;  
}
```

2.11 单链表按序号查找代码

顺着链表 next 指针往下找第 i 个节点。

```
LNode * GetElem(LinkList L, int i) {  
    if (i < 0) return NULL;  
    if (i == 0) return L; // 返回头结点  
  
    int j = 1;  
    LNode *p = L->next; // p 指向第一个数据节点  
    while (p != NULL && j < i) {  
        p = p->next;  
        j++;  
    }  
    return p;  
}
```

时间复杂度为 $O(n)$ 。

2.12 单链表的节点删除逻辑

假设已知待删除节点的前驱节点 p ，要删除 p 之后的节点 q 。

```
LNode *q = p->next; // 标记待删除节点  
p->next = q->next; // 从链表中断开 q  
free(q); // 释放内存空间
```

408 常见黑科技技巧：删除节点 $*p$ 本身 (无头指针情况)

若无法找到 p 的前驱节点，可采用**数据覆盖法**：将 p 后继节点的数据复制给 p ，然后删除后继节点。

代码： $q = p->next; p->data = q->data; p->next = q->next; free(q);$ (复杂度 $O(1)$)

| 2.13 双向链表 (Doubly Linked List) 定义

单链表只能单向遍历，双链表在每个节点增加了一个指向前驱的指针 prior。

```
typedef struct DNode {  
    ElemType data;  
    struct DNode *prior, *next; // 前驱和后继指针  
} DNode, *DLinklist;
```

双链表克服了单链表寻找前驱节点复杂度为 $O(n)$ 的缺点，在双链表中查找前驱为 $O(1)$ 。

2.14 双链表的插入核心代码

在双向链表中，在节点 p 之后插入节点 s ，顺序千万不能错。

```
// 1. 将  $s$  的后继指向  $p$  的后继  
s->next = p->next;  
  
// 2. 如果  $p$  不是最后一个节点, 将  $p$  的原后继的前驱指向  $s$   
if (p->next != NULL) {  
    p->next->prior = s;  
}  
  
// 3. 将  $s$  的前驱指向  $p$   
s->prior = p;  
  
// 4. 将  $p$  的后继指向  $s$   
p->next = s;
```

2.15 循环链表与静态链表

循环单链表

表中最后一个节点的指针不是 NULL，而是改为指向头结点，形成环。

判空条件：L->next == L

尾指针优势：若只设尾指针 R，则找尾节点和头节点的时间均为 $O(1)$ 。

静态链表

用数组来描述线性表的链式物理结构。指针域记录的是数组下标（称为“游标”）。

以 next == -1 作为其结束标志。

32. 【2016 统考真题】已知一个带有表头结点的循环双链表 L, 结点结构为

prev	data	next
------	------	------

, 其中 prev 和 next 分别是指向其直接前驱和直接后继结点的指针。现要删除指针 p 所指的结点, 正确的语句序列是 ()。

- A. `p->next->prev=p->prev; p->prev->next=p->prev; free(p);`
- B. `p->next->prev=p->next; p->prev->next=p->next; free(p);`
- C. `p->next->prev=p->next; p->prev->next=p->prev; free(p);`
- D. `p->next->prev=p->prev; p->prev->next=p->next; free(p);`

32. D

选项 A 的第二句代码, 相当于将 p 前驱结点的后继指针指向其自身, 错误; 选项 B 和 C 的第一句代码, 相当于将 p 后继结点的前驱指针指向其自身, 错误。只有选项 D 正确。

34. 【2021 统考真题】已知头指针 h 指向一个带头结点的非空循环单链表，结点结构为

data	next
------	------

，其中 $next$ 是指向直接后继结点的指针， p 是尾指针， q 是临时指针。现要删除该链表的第一个元素，正确的语句序列是（ ）。

- A. $h \rightarrow next = h \rightarrow next \rightarrow next; q = h \rightarrow next; free(q);$
- B. $q = h \rightarrow next; h \rightarrow next = h \rightarrow next \rightarrow next; free(q);$
- C. $q = h \rightarrow next; h \rightarrow next = q \rightarrow next; if (p \neq q) \ p = h; free(q);$
- D. $q = h \rightarrow next; h \rightarrow next = q \rightarrow next; if (p == q) \ p = h; free(q);$

34. D

如图 1 所示，要删除带头结点的非空循环单链表中的第一个元素，就要先用临时指针 q 指向待删结点， $q = h \rightarrow next$ ；然后将 q 从链表中断开， $h \rightarrow next = q \rightarrow next$ （这一步也可写成 $h \rightarrow next = h \rightarrow next \rightarrow next$ ）；此时要考虑一种特殊情况，若待删结点是链表的尾结点，即循环单链表中只有一个元素（ p 和 q 指向同一个结点），如图 2 所示，则在删除后要将尾指针指向头结点，即 $if (p == q) \ p = h$ ；最后释放 q 结点即可，答案选择选项 D。

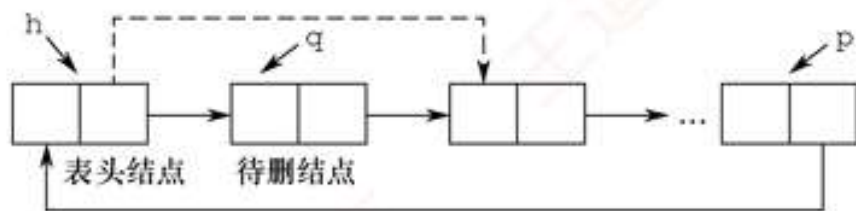


图 1

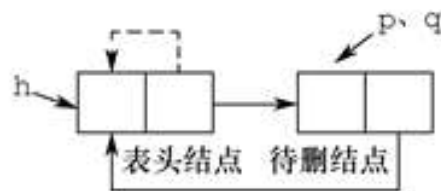


图 2

35. 【2023 统考真题】现有非空双向链表 L，其结点结构为

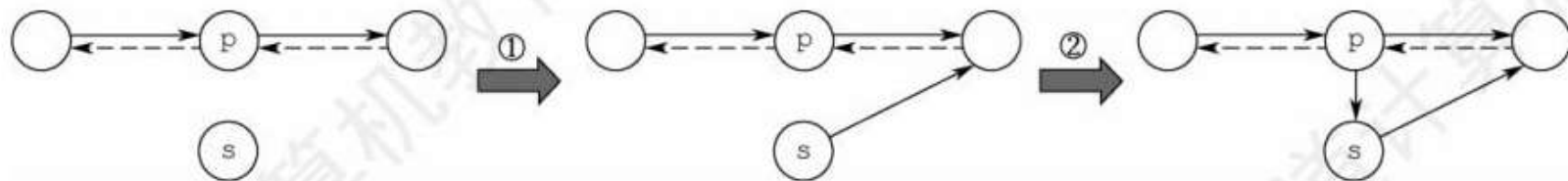
prev	data	next
------	------	------

，prev 是指向直接前驱结点的指针，next 是指向直接后继结点的指针。若要在 L 中指针 p 所指向的结点(非尾结点)之后插入指针 s 指向的新结点，则在执行语句序列“s->next=p->next; p->next=s;”后，下列语句序列中还需要执行的是()。

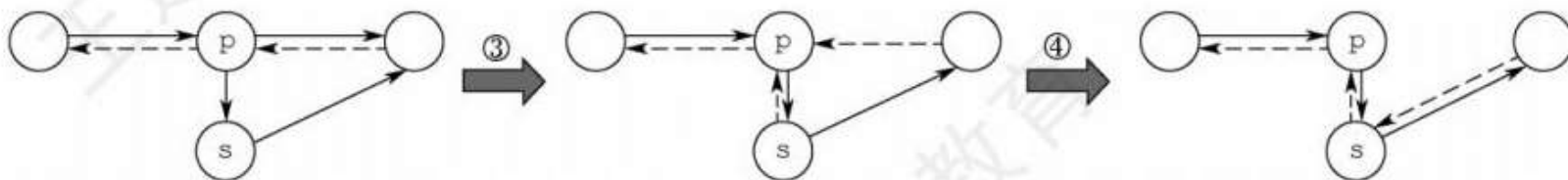
- A. s->next->prev=p; s->prev=p;
- B. p->next->prev=s; s->prev=p;
- C. s->prev=s->next->prev; s->next->prev=s;
- D. p->next->prev=s->prev; s->next->prev=p;

35. C

链表的插入操作要保证不会造成断链，画图再依次判断选项。执行完语句“①s->next=p->next; ②p->next=s;”后的结构如下图所示(虚线表示 prev，实线表示 next)。



对于 A, s->next->prev=p, 错误。对于 B, p->next->prev=s, 让 s 的 prev 指向 s, 错误。对于 D, 两句代码均错误。对于 C, 执行完语句“③s->prev=s->next->prev; ④s->next->prev=s;”后的结构如下图所示, 满足插入的要求。



| 3.1 栈 (Stack) 的基本概念

栈是只允许在一端进行插入或删除操作的线性表。

- **栈顶 (Top):** 线性表允许进行插入删除的那一端。
- **栈底 (Bottom):** 固定的, 不允许进行插入和删除的另一端。
- **特性:** 后进先出 (LIFO - Last In First Out)。

考点数学公式: n 个不同元素进栈, 出栈序列的合法个数为卡特兰数: $\frac{1}{n+1} C_{2n}^n$

3.2 顺序栈结构与初始化代码

利用一组地址连续的存储单元存放栈的数据，用 top 记录栈顶。

```
#define MaxSize 50
typedef struct {
    ElemType data[MaxSize];
    int top; // 栈顶指针 (即数组下标)
} SqStack;

void InitStack(SqStack &S) {
    S.top = -1; // 初始化栈顶指针为 -1 (重要习惯)
}
```

判栈空条件：S.top == -1。判栈满条件：S.top == MaxSize - 1。

3.3 顺序栈：进栈与出栈代码

Push 进栈

```
bool Push(SqStack &S, ElemType x) {  
    if (S.top == MaxSize-1)  
        return false; // 满栈报错  
    S.data[++S.top] = x; // 指针先加1, 再入栈  
    return true;  
}
```

Pop 出栈

```
bool Pop(SqStack &S, ElemType &x) {  
    if (S.top == -1)  
        return false; // 空栈报错  
    x = S.data[S.top--]; // 先出栈, 指针再减1  
    return true;  
}
```

细节：如果初始化时 $S.top = 0$ ，则入栈变为 $S.data[S.top++] = x$ ；，判断条件也会随之改变。

| 3.4 共享栈 (Shared Stack)

为了提高内存利用率，让两个顺序栈共享一个一维数组空间。两个栈顶指针向中间延伸。

- 初始化: $\text{top0} = -1$, $\text{top1} = \text{MaxSize}$
- 进栈 0: $\text{top0}++$; 进栈 1: $\text{top1}--$
- 共享栈满的条件: $\text{top0} + 1 == \text{top1}$

此设计仅当两个栈的需求互相补充时效果最好，极大降低了栈溢出的概率。

| 3.5 队列 (Queue) 的基本概念

队列也是一种操作受限的线性表，只允许在表的一端进行插入，而在表的另一端进行删除。

- **队头 (Front)**: 允许删除的一端。
- **队尾 (Rear)**: 允许插入的一端。
- **特性**: 先进先出 (FIFO - First In First Out)。

应用场景: 操作系统中的作业排队、缓冲区管理（如键盘输入缓冲区）、广度优先遍历 (BFS)。

3.6 循环队列 (Circular Queue) 结构

普通顺序队列会出现“假溢出”现象。为解决此问题，将队列逻辑上视为一个环，利用模运算 % 实现。

```
#define MaxSize 50
typedef struct {
    ElemType data[MaxSize];
    int front, rear; // 队头指针和队尾指针
} SqQueue;

void InitQueue(SqQueue &Q) {
    Q.rear = Q.front = 0;
}
```

| 3.7 循环队列判满的三种方式 (408重点)

因为初始 $\text{front} == \text{rear}$ ，队满时也会 $\text{front} == \text{rear}$ ，所以必须区分。

1 牺牲一个存储单元 (最常见):

队满条件: $(\text{rear} + 1) \% \text{MaxSize} == \text{front}$

队列长度: $(\text{rear} - \text{front} + \text{MaxSize}) \% \text{MaxSize}$

2 结构体增设 size 变量:

队空: $\text{size} == 0$; 队满: $\text{size} == \text{MaxSize}$ 。出入队时维护 size。

3 结构体增设 tag 变量:

插入成功置 $\text{tag}=1$ ，删除成功置 $\text{tag}=0$ 。只有 $\text{tag}=1$ 且 $\text{front}==\text{rear}$ 时才是满。

3.8 循环队列：入队与出队代码

EnQueue 入队

```
bool EnQueue(SqQueue &Q, ElemType x) {  
    // 采用牺牲一个单元法判满  
    if ((Q.rear + 1) % MaxSize == Q.front)  
        return false;  
    Q.data[Q.rear] = x;  
    Q.rear = (Q.rear + 1) % MaxSize;  
    return true;  
}
```

DeQueue 出队

```
bool DeQueue(SqQueue &Q, ElemType &x) {  
    if (Q.rear == Q.front)  
        return false; // 队空报错  
    x = Q.data[Q.front];  
    Q.front = (Q.front + 1) % MaxSize;  
    return true;  
}
```

3.9 链式队列代码解析

链式队列就是一个同时带有队头指针和队尾指针的单链表。

```
typedef struct LinkNode { // 链式队列节点
    ElemType data;
    struct LinkNode *next;
} LinkNode;

typedef struct { // 链式队列管理
    LinkNode *front, *rear;
} LinkQueue;
```

入队操作相当于单链表的**尾插法**，出队操作相当于单链表的**头节点删除**。

队列的变种

双端队列 ⊖ 允许从两端插入、两端删除的队列

输入受限的双端队列 ⊖ 允许从两端删除、从一端插入的队列

输出受限的双端队列 ⊖ 允许从两端插入、从一端删除的队列

考点：对输出序列合法性的判断

在栈中合法的输出序列，在双端队列中必定合法

3.10 栈的经典应用：括号匹配算法思路

算法思想（408常见大题/选择题原型）：

1. 初始化一个空栈。顺序扫描给定字符串。
2. 遇到左括号 (、[、{，直接压入栈中。
3. 遇到右括号，检查栈是否为空：
 - 若为空，则说明右括号多余，匹配失败。
 - 若不为空，弹出栈顶元素，检查左右括号类型是否匹配。
4. 扫描结束后，若栈为空，则匹配成功；否则左括号多余，匹配失败。

表达式求值问题

概念 ⊖ 运算符、操作数、界限符 (DIY概念: 左操作数/右操作数)

三种表达式 ⊖
中缀表达式 ⊖ 运算符在操作数中间
后缀表达式 (逆波兰式) ⊖ 运算符在操作数后面
前缀表达式 (波兰式) ⊖ 运算符在操作数前面

一个中缀表达式可以对应多个后缀、前缀表达式

一个中缀表达式只对应一个后缀表达式 (确保算法的“确定性”)



后缀表达式考点

中缀转后缀

①按“左优先”原则确定运算符的运算次序

②根据①中确定的次序, 依次将各个运算符和与之相邻的两个操作数按<左操作数 右操作数 运算符>的规则合体

后缀转中缀

从左往右扫描, 每遇到一个运算符, 就将<左操作数 右操作数 运算符>变为(左操作数 运算符 右操作数)的形式

计算

从左往右扫描, 遇到操作数入栈, 遇到运算符则弹出两个栈顶元素运算后入栈 (注意: 先弹出的元素是“右操作数”)

前缀表达式

中缀转前缀

①按“右优先”原则确定运算符的运算次序

②根据①中确定的次序, 依次将各个运算符和与之相邻的两个操作数按<运算符 左操作数 右操作数>的规则合体

计算

从右往左扫描, 遇到操作数入栈, 遇到运算符则弹出两个栈顶元素运算后入栈 (注意: 先弹出的元素是“左操作数”)

王道考研/CSKAOYAN.COM

用栈实现中缀表达式转后缀表达式:

初始化一个栈, 用于保存暂时还不能确定运算顺序的运算符。

从左到右处理各个元素, 直到末尾。可能遇到三种情况:

- ① 遇到操作数。直接加入后缀表达式。
- ② 遇到界限符。遇到“(”直接入栈; 遇到“)”则依次弹出栈内运算符并加入后缀表达式, 直到弹出“(”为止。注意:“(”不加入后缀表达式。
- ③ 遇到运算符。依次弹出栈中优先级高于或等于当前运算符的所有运算符, 并加入后缀表达式, 若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后, 将栈中剩余运算符依次弹出, 并加入后缀表达式。

用栈实现后缀表达式的计算:

- ① 从左往右扫描下一个元素, 直到处理完所有元素
- ② 若扫描到操作数则压入栈, 并回到①; 否则执行③
- ③ 若扫描到运算符, 则弹出两个栈顶元素, 执行相应运算, 运算结果压回栈顶, 回到①

用栈实现中缀表达式的计算:

初始化两个栈, 操作数栈和运算符栈

若扫描到操作数, 压入操作数栈

若扫描到运算符或界限符, 则按照“中缀转后缀”相同的逻辑压入运算符栈(期间也会弹出运算符, 每当弹出一个运算符时, 就需要再弹出两个操作数栈的栈顶元素并执行相应运算, 运算结果再压回操作数栈)

3.11 多维数组的存储地址映射

多维数组映射到一维连续空间，通常分为“行优先”和“列优先”。

行优先存储公式 (下标从 0 开始)

对于二维数组 $A[M][N]$ ：

$$Loc(i, j) = Loc(0, 0) + (i \times N + j) \times L$$

其中 L 为每个元素占用的字节数， N 为总列数。

注意防坑：408 题目中有时下标从 1 开始定义矩阵 $A[1...M][1...N]$ ，此时公式需变为 $(i - 1) \times N + (j - 1)$ 。

3.12 特殊矩阵的压缩存储

将重复出现的非零元素或零元素剔除，节省空间。通常映射到一维数组 $B[k]$ 中。

矩阵类型	存储策略	一维数组长度
对称矩阵	只存下三角（或上三角）及主对角线	$\frac{n(n+1)}{2}$
三角矩阵	同上，最后加一个元素存常数 c	$\frac{n(n+1)}{2} + 1$
三对角矩阵	只存主对角线及其上下相邻对角线（ $ i-j \leq 1$ ）	$3n - 2$

30. 【2020 统考真题】对空栈 S 进行 Push 和 Pop 操作，入栈序列为 a, b, c, d, e ，经过 Push、Push、Pop、Push、Pop、Push、Push、Pop 操作后得到的出栈序列是 ()。

- A. b, a, c B. b, a, e C. b, c, a D. b, c, e

30. D

按题意，出入栈操作的过程如下：

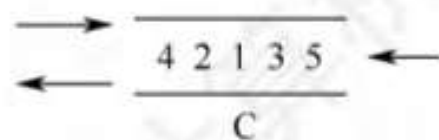
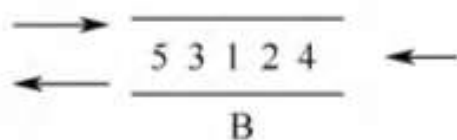
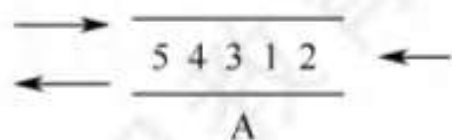
操作	栈内元素	出栈元素
Push	a	
Push	$a b$	
Pop	a	b
Push	$a c$	
Pop	a	c
Push	$a d$	
Push	$a d e$	
Pop	$a d$	e

24. 【2021 统考真题】初始为空的队列 Q 的一端仅能进行入队操作，另外一端既能进行入队操作又能进行出队操作。若 Q 的入队序列是 1, 2, 3, 4, 5，则不能得到的出队序列是()。

- A. 5, 4, 3, 1, 2 B. 5, 3, 1, 2, 4 C. 4, 2, 1, 3, 5 D. 4, 1, 3, 2, 5

24. D

假设队列左端允许入队和出队，右端只能入队。对于 A，依次从右端入队 1, 2，再从左端入队 3, 4, 5。对于 B，从右端入队 1, 2，然后从左端入队 3，再从右端入队 4，最后从左端入队 5。对于 C，从左端入队 1, 2，然后从右端入队 3，再从左端入队 4，最后从右端入队 5。无法验证 D 的序列。



18. 【2018 统考真题】若栈 S1 中保存整数，栈 S2 中保存运算符，函数 F() 依次执行下述各步操作：

- 1) 从 S1 中依次弹出两个操作数 a 和 b。
- 2) 从 S2 中弹出一个运算符 op。
- 3) 执行相应的运算 $b \text{ op } a$ 。
- 4) 将运算结果压入 S1 中。

假定 S1 中的操作数依次是 5, 8, 3, 2 (2 在栈顶)，S2 中的运算符依次是 *, -, + (+ 在栈顶)。调用 3 次 F() 后，S1 栈顶保存的值是 ()。

- A. -15 B. 15 C. -20 D. 20

18. B

第一次调用：①从 S1 中弹出 2 和 3；②从 S2 中弹出+；③执行 $3+2=5$ ；④将 5 压入 S1 中，第一次调用结束后 S1 中剩余 5、8、5 (5 在栈顶)，S2 中剩余*、- (-在栈顶)。第二次调用：①从 S1 中弹出 5 和 8；②从 S2 中弹出-；③执行 $8-5=3$ ；④将 3 压入 S1 中，第二次调用结束后 S1 中剩余 5、3 (3 在栈顶)，S2 中剩余*。第三次调用：①从 S1 中弹出 3 和 5；②从 S2 中弹出*；③执行 $5*3=15$ ；④将 15 压入 S1 中，第三次调用结束后 S1 中仅剩余 15，S2 为空。

31. 【2022 统考真题】给定有限符号集 S , in 和 out 均为 S 中所有元素的任意排列。对于初始为空的栈 ST , 下列叙述中, 正确的是 ()。
- A. 若 in 是 ST 的入栈序列, 则不能判断 out 是否为其可能的出栈序列
 - B. 若 out 是 ST 的出栈序列, 则不能判断 in 是否为其可能的入栈序列
 - C. 若 in 是 ST 的入栈序列, out 是对应 in 的出栈序列, 则 in 与 out 一定不同
 - D. 若 in 是 ST 的入栈序列, out 是对应 in 的出栈序列, 则 in 与 out 可能互为倒序

31. D

通过模拟出入栈操作, 可以判断入栈序列 in 和出栈序列 out 是否合法。因此, 已知 in 序列可以判断 out 序列是否为可能的出栈序列; 已知 out 序列也可以判断 in 序列是否为可能的入栈序列, 选项 A 和 B 错误。若每个元素入栈后立即出栈, 则 in 序列和 out 序列相同, 选项 C 错误。若所有元素都入栈后才依次出栈, 则 in 序列和 out 序列互为倒序, 选项 D 正确。

15. 【2021 统考真题】二维数组 A 按行优先方式存储, 每个元素占用 1 个存储单元。若元素 $A[0][0]$ 的存储地址是 100, $A[3][3]$ 的存储地址是 220, 则元素 $A[5][5]$ 的存储地址是 ()。
- A. 295 B. 300 C. 301 D. 306

15. B

二维数组 A 按行优先存储, 每个元素占用 1 个存储单元, 由 $A[0][0]$ 和 $A[3][3]$ 的存储地址可知 $A[3][3]$ 是二维数组 A 中的第 121 个元素, 假设二维数组 A 的每行有 n 个元素, 则 $n \times 3 + 4 = 121$, 求得 $n = 39$, 所以元素 $A[5][5]$ 的存储地址为 $100 + 39 \times 5 + 6 - 1 = 300$ 。

14. 【2020 统考真题】将一个 10×10 阶对称矩阵 M 的上三角部分的元素 $m_{i,j}$ ($1 \leq i \leq j \leq 10$) 按列优先存入 C 语言的一维数组 N 中, 元素 $m_{7,2}$ 在 N 中的下标是 ()。
- A. 15 B. 16 C. 22 D. 23

14. C

上三角矩阵按列优先存储, 先存储只有 1 个元素的第一列, 再存储有 2 个元素的第二列, 以此类推。 $m_{7,2}$ 位于左下角, 对应右上角的元素为 $m_{2,7}$, 在 $m_{2,7}$ 之前存有

第 1 列: 1

第 2 列: 2

.....

第 6 列: 6

第 7 列: 1

前面共存储有 $1 + 2 + 3 + 4 + 5 + 6 + 1 = 22$ 个元素 (数组下标范围为 $0 \sim 21$), 注意数组下标从 0 开始, 所以 $m_{2,7}$ 在数组 N 中的下标为 22, 即 $m_{7,2}$ 在数组 N 中的下标为 22。

19. 【2024 统考真题】与表达式 $x+y*(z-u)/v$ 等价的后缀表达式是 ()。

A. $xyzuv-*v/+$

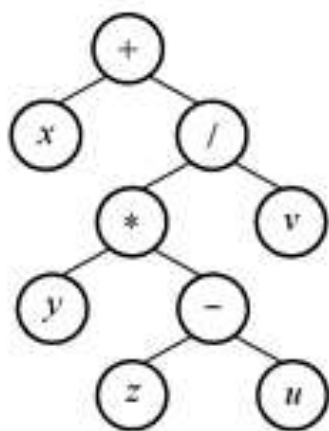
B. $xyzuv-v/*+$

C. $+x/*y-zuv$

D. $+x*y/-zuv$

19. A

根据中缀表达式画出对应的二叉树如右图所示，对该二叉树进行后序遍历即可得到后缀表达式为 $xyzuv-*v/+$ 。本题也可采用本书前面描述的手算方法。



参考

王道计算机数据结构考研复习指导

MIT 6.006 Introduction to Algorithms (Fall 2011)

2026年计算机学科专业基础考试大纲